

APDS7311w POE 1

Secure International Payments Portal

Full Assignment Report and Implementation Notes

Field	Details
Module name	Application Development Security
Module code	APDS7311w
Student name	
Student number	
Project	Customer and Employee International Payments Portal
Technology used	React, Node.js API, SQLite, HTTPS, bcrypt, JWT cookies, SonarQube/CircleCI config

Important note for submission: run the code, record your own demo video, and add your own MobSF/ScoutSuite screenshots. The code and written report are included in this package.

1. Introduction

This POE solution is for a secure international payments system. The system has two parts. The first part is for customers to register, log in and capture an international payment. The second part is for bank employees to log in, view the payment, verify the account and SWIFT details, and then send the transaction to a SWIFT handoff screen.

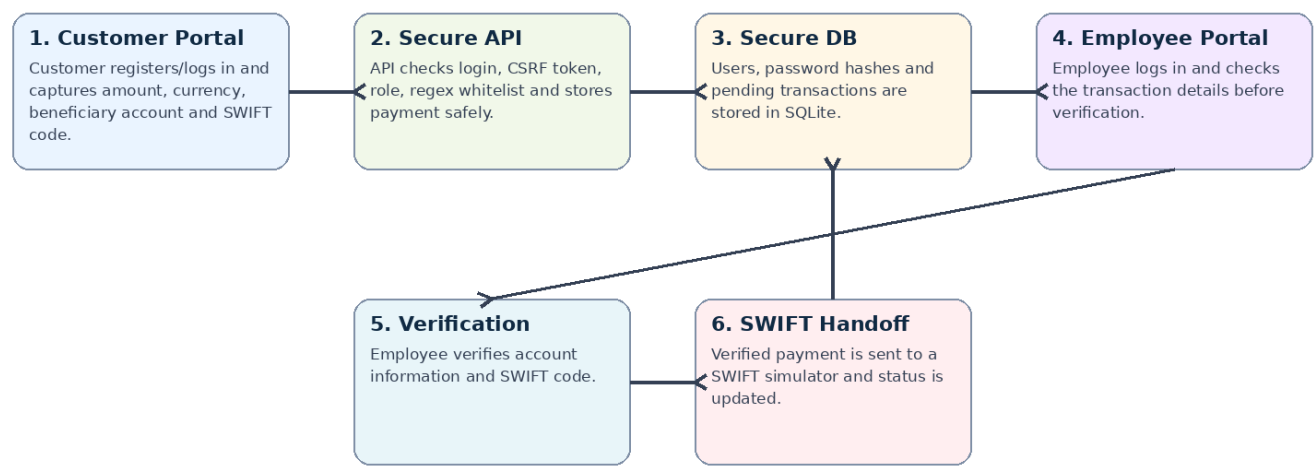
The solution focuses on security from the design stage and also in the actual code. The main security areas are password hashing and salting, whitelist validation, HTTPS, secure cookies, role-based access, CSRF protection, rate limiting, clickjacking protection, XSS reduction and SQL injection prevention.

2. Task 1: Solution Architecture and Security Planning

2.1 Data flow in the system

The data flow starts when the customer uses the customer portal. The payment only reaches the SWIFT handoff after a bank employee has logged in and verified the transaction. The flow is shown below.

International Payment Data Flow



Security points: HTTPS for traffic, bcrypt password hashing, secure cookie, CSRF header, rate limits, Helmet headers, role-based access, and parameterised queries.

2.2 Data flow steps

Step	What happens	Security used
1	Customer registers with full name, ID number, account number and password.	Regex whitelist checks, password strength rule, bcrypt hashing and salting.
2	Customer logs in using account number and password.	bcrypt password comparison, login rate limit and secure session cookie.
3	Customer captures payment amount, currency, provider, beneficiary account, bank name and SWIFT code.	Whitelist input checks, no raw HTML, amount limits and allowed currency/provider lists.

4	API stores transaction as PENDING.	Parameterised queries and database constraints.
5	Employee logs into the internal portal.	Employee account is pre-created. No public employee registration is allowed.
6	Employee verifies beneficiary account and SWIFT code.	Role-based access control. Only EMPLOYEE role can verify.
7	Verified transaction is sent to SWIFT simulator.	Only VERIFIED payments can be sent. Status changes to SENT_TO_SWIFT.

2.3 How input information is secured

- All important fields are checked using whitelist regular expressions. The application does not try to remove bad characters only; it only accepts known good formats.
- Customer ID number must be 13 digits and account numbers must only contain digits.
- SWIFT code must match the normal 8 or 11 character SWIFT/BIC format.
- Password must be between 10 and 64 characters and must contain uppercase, lowercase, number and special character.
- The API also validates input again. The front-end checks are only for user experience. The server-side checks are the real security control.

2.4 How data in transit is secured

- The API runs on HTTPS at https://localhost:5001 for development.
- The React portal runs on HTTPS at https://localhost:5173 for development.
- The session cookie is marked Secure, so the browser should only send it over HTTPS.
- In production, the portal should use a trusted TLS certificate and HSTS should be enabled.

2.5 Hardening against required attacks

Security Hardening Used in the Portal

Session hijacking

HttpOnly Secure cookie, short JWT lifetime, SameSite Strict, logout clears session.

Clickjacking

Helmet frameguard deny and CSP frame-ancestors none.

SQL injection

Regex whitelist validation and parameterised database queries.

XSS

Whitelist input, no raw HTML, React escaping, CSP and no-sniff headers.

Man in the Middle

HTTPS, HSTS in production, secure cookies only over TLS.

DoS / brute force

Global and login rate limits, small JSON body size, simple endpoints.

Attack	Risk	Control used in this solution
Session hijacking	Attacker steals or reuses a user session.	HttpOnly Secure SameSite cookie, short JWT expiry, logout clears cookie, no token stored in localStorage.
Clickjacking	Portal is loaded in an invisible frame and user is tricked into clicking.	Helmet frameguard deny and CSP frame-ancestors none.
SQL injection	Attacker changes SQL commands using input fields.	Regex whitelist validation and parameterised database statements.
Cross-site scripting	Attacker injects JavaScript into the page.	React escapes output, no raw HTML, whitelist validation and CSP/no-sniff headers.
Man in the Middle	Attacker reads or changes traffic between browser and server.	HTTPS, secure cookies, production HSTS recommendation.
DoS attacks	System is flooded with requests or login attempts.	Global rate limit, login rate limit, small JSON request body limit and simple API responses.

2.6 Security tool plan and short report

The brief asks for tools such as MobSF and ScoutSuite. These tools must be run on the actual mobile app and AWS account supplied by the lecturer. I cannot run them without the APK/IPA file and AWS credentials, so this section is written as a ready-to-use report structure that can be completed with the actual screenshots and results.

Tool	Purpose	How it fits this project	My recommendation
MobSF	Static/dynamic security scan for mobile apps.	Used to test the semester one mobile app and check for insecure storage, weak network settings, risky permissions and hardcoded secrets.	Use it before mobile releases because it gives mobile-specific findings that normal web scanners may miss.
ScoutSuite	Cloud security auditing for AWS/Azure/GCP.	Used to review the AWS hosting environment for risky services, public buckets, weak IAM, exposed ports and logging gaps.	Use it before deployment and after major AWS changes. It is useful for change review meetings.
AWS CLI	Command line access to AWS.	Needed so ScoutSuite can authenticate and inspect the provided AWS account.	Use a least-privilege read-only audit account. Never commit credentials.
SonarQube	Code quality, code smells and security hotspots.	Configured through CircleCI and sonar-project.properties in this package.	Use it on every pull request to find risky code early.

2.7 Example MobSF report wording

After running MobSF, I would include the score, screenshots and high-risk findings. A short student-style finding could be:

Sample wording: MobSF was useful because it highlighted security areas that are easy to miss in a mobile app, such as unsafe network configuration, insecure storage and risky permissions. If the scan shows hardcoded keys or clear-text traffic, those issues must be fixed before the mobile app is accepted by the security team. I would support using MobSF as part of the CR process because it gives repeatable evidence for mobile app security checks.

2.8 Example ScoutSuite report wording

Sample wording: ScoutSuite should be used on the provided AWS account to check if the hosting environment is secure. The most important areas to check are IAM permissions, public access, security groups, encryption and logging. If ScoutSuite finds public storage or overly open ports, those items must be fixed before the portal is hosted. I support using ScoutSuite because it checks the cloud environment and not only the application code.

3. Task 2: Secure Customer International Payments Portal

3.1 What was built

The customer portal is built in React and talks to the secure API. A customer can register, log in, capture an international payment and view the payment status. The API stores the payment as PENDING until an employee verifies it.

Requirement from brief	Where it is implemented
Password security with hashing and salting	api/src/routes.js uses bcrypt.hashSync for registration and bcrypt.compareSync for login.
Whitelist all input using RegEx patterns	api/src/validators.js and web/src/lib/validators.js contain whitelist regex patterns.
All traffic served over SSL	API and React dev server both use HTTPS with the included localhost certificate.
Protect against attacks	api/src/middleware/security.js and api/src/middleware/auth.js include rate limits, Helmet, CSRF and secure cookies.
Video evidence	Record the suggested demo flow in README.md using OBS and upload as unlisted YouTube video.

3.2 Customer functions

1. Register with full name, ID number, account number and password.
2. Log in using account number and password.
3. Capture amount, currency, payment provider, beneficiary name, beneficiary account, bank name and SWIFT code.
4. Click Pay Now to save the payment.
5. View the payment status on the dashboard.

3.3 Customer API endpoints

Method	Endpoint	Purpose	Security
POST	/api/auth/register-customer	Creates customer account and starts session.	Regex validation, bcrypt hash, login rate limit.
POST	/api/auth/login-customer	Logs customer in.	Regex validation, bcrypt compare, secure cookie.
GET	/api/auth/me	Gets current logged in user.	Requires valid session.
POST	/api/payments	Creates payment.	Customer role required and CSRF token required.
GET	/api/payments/my	Lists customer payments.	Customer role required.

3.4 Database tables used

Table	Main fields	Purpose
users	id, role, full_name, id_number,	Stores customer and employee

	account_number, password_hash, created_at	accounts. Passwords are stored as hashes only.
payments	id, customer_id, amount_cents, currency, provider, beneficiary_name, beneficiary_account, swift_code, bank_name, status, employee_id, verified_at, sent_to_swift_at	Stores captured international payments and status changes.

4. Task 3: Secure Employee Internal Payments Portal

4.1 What was built

The employee internal portal is built in the same React application, but it uses a different login and role. Employees cannot self-register. The API seeds one employee account for marking and demo purposes. The employee can view payments, verify them and send verified payments to the SWIFT simulator.

Requirement from brief	Where it is implemented
Users are created, no registration possible	Only customer registration route exists. Employee is seeded in <code>api/src/db/database.js</code> .
Password hashing and salting	Seeded employee password is hashed with <code>bcrypt</code> .
Whitelist input using RegEx patterns	Employee username must match EMP followed by digits. Payment IDs are numeric only.
All traffic served over SSL	Employee portal and API run over HTTPS.
Protect against attacks listed in description	Same API security middleware protects employee endpoints. Role checks prevent customer access.
GitHub and SonarQube/CircleCI	The package contains <code>.circleci/config.yml</code> and <code>sonar-project.properties</code> .
Video evidence	Record the employee workflow in OBS and upload as unlisted YouTube video.

4.2 Employee functions

- Employee logs in with the pre-created employee account.
- Employee views customer transactions waiting for verification.
- Employee checks beneficiary account number, bank name and SWIFT code.
- Employee clicks Verify to change status from PENDING to VERIFIED.
- Employee clicks Open SWIFT and send to change status from VERIFIED to SENT_TO_SWIFT.

4.3 Employee API endpoints

Method	Endpoint	Purpose	Security
POST	<code>/api/auth/login-employee</code>	Logs employee in.	Regex validation, <code>bcrypt</code> compare, secure cookie.
GET	<code>/api/employee/payments</code>	Lists payments for employee review.	Employee role required.
POST	<code>/api/employee/payments/:id/verify</code>	Verifies pending transaction.	Employee role and CSRF token required.
POST	<code>/api/employee/payments/:id/send-to-swift</code>	Sends verified payment to SWIFT simulator.	Employee role and CSRF token required.

4.4 SonarQube and CircleCI setup

A CircleCI configuration is included in `.circleci/config.yml`. It installs the API and web packages, builds the React app and then runs a SonarQube scan when `SONAR_TOKEN` and `SONAR_HOST_URL` are configured in CircleCI project settings. The SonarQube project settings are in `sonar-project.properties`.

- Push the project to GitHub.
- Connect the GitHub repo to CircleCI.
- Create a SonarQube/SonarCloud project and copy the token.
- Add SONAR_TOKEN and SONAR_HOST_URL as environment variables in CircleCI.
- Commit and push again so CircleCI can run the scan.

5. How to Run the Project

Step	Command or action
1	Open the project folder.
2	Run API: cd api, then npm install, then npm run dev.
3	Run React: cd web, then npm install, then npm run dev.
4	Open https://localhost:5173 in the browser.
5	If the browser warns about the certificate, accept it for local testing only.
6	Use employee demo login EMP001 / Employee@12345.

6. Testing Checklist

Test	Expected result
Register customer with valid data	Customer is created and logged in.
Register with invalid ID number	API rejects it because it is not 13 digits.
Login with wrong password	API rejects it.
Capture valid payment	Payment is saved as PENDING.
Try invalid SWIFT code	API rejects it.
Login as employee	Employee dashboard opens.
Verify pending payment	Status changes to VERIFIED.
Send verified payment to SWIFT	Status changes to SENT_TO_SWIFT and a SWIFT reference is shown.
Try employee page as customer	Access is denied by role-based API checks.

7. Video Checklist for OBS

- Show both terminals running: API and React app.
- Show the browser at https://localhost:5173.
- Register a customer and mention password validation.
- Capture a payment and show the SWIFT code field.
- Login as employee and verify the same payment.
- Click Open SWIFT and send, then show the final status.
- Briefly show the code files for validators, security middleware and database seeding.
- Upload the video as unlisted to YouTube and put the link in your submission.

8. Files Included in the Submission Package

Folder/File	Purpose
api/	Secure backend API.
web/	React portal for customer and employee workflows.
docs/	This report in Word and PDF.
diagrams/	Mermaid diagrams for architecture and data flow.
.circleci/config.yml	CircleCI pipeline with SonarQube scan step.

sonar-project.properties	SonarQube project configuration.
README.md	Run instructions and demo user details.

9. Conclusion

The completed solution covers the main requirements in the POE brief. It includes a customer portal, employee internal portal, secure API, database storage, HTTPS, password hashing and salting, whitelist input validation and protections against the listed attacks. The tool section also explains how MobSF, ScoutSuite, AWS CLI and SonarQube fit into the security process.